System Architecture — AI-сервис генерации презентаций для российских вузов (B2B2C)

Версия: 1.0
Дата: 28.02.2026
Команда: Ангелы Бизнеса

Документ описывает целевую архитектуру MVP сервиса генерации презентаций, ориентированную на SSO, brand-lock, частичную регенерацию, экспорт PPTX/PDF, админ-контур (дашборды/аудит) и комплаенс по ПДн.

Оглавление

1. C4 L1 — Context 4
   1.1 Акторы и внешние системы 4
   1.2 Диаграмма контекста (ASCII) 4
2. C4 L2 — Containers 5
   Web App (UI) 5
      Ответственность: 5
      Ключевые функции: 5
   Orchestrator API (Backend) 5
      Ответственность: 5
      Ключевые функции: 5
   LLM Gateway 5
      Ответственность: 5
   Storage (DB + Object Storage) 5
      Ответственность: 5
   Renderer (PPTX/PDF) 6
      Ответственность: 6
   Asset Service 6
      Ответственность: 6
   Observability (Logs/Metrics/Traces + Audit) 6
      Ответственность: 6
   2.8 Диаграмма контейнеров (ASCII) 6
3. Основные sequence flows 7
   3.1 Flow A — Generate 7
      Принцип anti-hallucination 8
   3.2 Flow B — Edit (частичная регенерация) 8
   3.3 Flow C — Export (PPTX/PDF) 8
4. Выбор технологий (варианты + почему) 8
5. Хранение и версионирование промптов 9
   Хранение 9
   Версионирование 9
   Воспроизводимость 9
   Контроль 9

6. Безопасность (PII/ПДн, секреты, RBAC, audit log) 9
PII/ПДн 9
Secrets 9
RBAC 9
Audit log 9
7. Масштабирование (MVP → growth) 9

1. C4 L1 — Context

1.1 Акторы и внешние системы

Актор/Система	Роль во взаимодействии
Student / Teacher	Создают, правят и экспортируют презентации.
Admin (ВУЗ)	Управляет шаблонами/брендбуком, ролями, отчётами.
Buyer	Оценивает демо/документы для закупки (может совпадать с Admin).
University SSO/IdP	Аутентификация (SAML/OIDC).
LLM Provider(s)	Модели (через LLM Gateway).
Storage (DB + Object Storage)	Хранение сущностей и файлов (источники/экспорты/артефакты).
Observability stack	Метрики, логи, трассировка, аудит.

1.2 Диаграмма контекста (ASCII)

```
+-------------------+      +-------------------------+
| Student / Teacher |<------>|  Web App (UI)          |
+-------------------+      +-----------+-------------+
                             |
                             v
+-------------------+      +-------------------------+
| Admin / Buyer     |<------>|  API / Orchestrator     |
+-------------------+      +-----+---------+---------+
                         |       |
                         |       v
                         |  +-----+--------+
                         |  | LLM Gateway   |<----> LLM Provider(s)
                         |  +--------------+
                         |
                         v
                    +--------+---------+
                    | Storage (DB+S3)  |
                    +--------+---------+
                         |
                         v
                    +--------+---------+
                    | Renderer (PPTX)  |
                    +--------+---------+
                         |
                         v
                     Export (PPTX/PDF)

University SSO/IdP <----> UI/Orchestrator (auth)
Observability stack <---- all services (metrics/logs/traces/audit)
```

2. C4 L2 — Containers

Web App (UI)

Ответственность:

- SSO-вход, работа с brief, запуск генерации/правок/экспорта, просмотр статусов.
- Редактор слайдов с brand-locked controls.
- Admin-режим: загрузка брендбука/шаблонов, управление орг-структурой, дашборды.

Ключевые функции:

- Draft workspace (deck editor)
- Progress UI (generation/export)
- Admin console (templates, roles, dashboards)

Orchestrator API (Backend)

Ответственность:

- Управляет жизненным циклом Deck / Generation Run / Export Job.
- Оркестрирует пайплайн (normalize → outline → copy → QA → renderSpec).
- Поддерживает частичную регенерацию (per-slide/per-block).
- Применяет политики: brand-lock, лимиты, идемпотентность, ретраи.
- API для UI и интеграция с SSO.

Ключевые функции:

- Auth/RBAC module
- Deck service (CRUD, versions)
- Generation service (runs, steps, caches)
- Export service (jobs, files)
- Admin service (templates, brand tokens)
- Metrics/Audit service

LLM Gateway

Ответственность:

- Единая точка вызова LLM: routing (cheap → strong), fallback.
- Лимиты токенов/стоимости, кэширование.
- Structured output: JSON schema validation.
- Redaction/PII-маскирование в логах.
- Retry/backoff на временные ошибки.

Storage (DB + Object Storage)

Ответственность:

- PostgreSQL: Tenants, Users, Roles, OrgUnits; Decks, Slides, Versions; GenerationRuns/StepResults; Exports/Jobs; AuditEvents; MetricsAgg.

- S3-совместимое хранилище (MinIO): источники, шаблоны/ассеты, экспорты, промежуточные JSON-артефакты.
- Шрифты — только при наличии лицензии и политики вуза.

Renderer (PPTX/PDF)

Ответственность:

- Сборка финального файла по render_spec: применение шаблонов/макетов, размещение элементов, генерация PPTX/PDF.
- Рекомендуемая реализация: stateless workers + очередь задач.

Asset Service

Ответственность:

- Подготовка ассетов: иконки, графики из таблиц (если есть данные), безопасная обработка изображений (resize/compress).
- MVP: каталог иконок + placeholders; графики только из явно переданных таблиц.

Observability (Logs/Metrics/Traces + Audit)

Ответственность:

- Технические метрики: latency, errors, cost.
- Трассировка шагов пайплайна.
- Audit log (неизменяемые события).
- Продуктовые метрики: BPC, activation, exports, time-to-first-deck.

2.8 Диаграмма контейнеров (ASCII)

```
+---------------------------+
|      Web App (UI)       |
| - Brief/Deck editor      |
| - Admin console          |
| - Progress UI            |
+-------------+-------------+
           |
           v
+---------------------------+     +------------------+
|    Orchestrator API      |<---->| University SSO   |
| - Auth/RBAC              |      | (SAML/OIDC IdP)  |
| - Deck/Run/Export        |      +------------------+
| - Policies (brand-lock)  |
+------+------+------+------+
      |      |      |
      |      |      v
      |      |   +------------------+
      |      |   | Renderer Service |
```

```
    |     |   | (PPTX/PDF)      |
    |     |   +------------------+
    |     |
    |     v
    |   +------------------+       +------------------+
    |   |   LLM Gateway     |<------>| LLM Provider(s)  |
    |   | (routing/cost/    |       +------------------+
    |   |  schema/PII)      |
    |   +------------------+
    |
    v
+---------------------------+       +------------------+
| Storage                   |<------>| Asset Service    |
| - Postgres (entities)     |        | (icons/charts)   |
| - Object storage (S3)     |        +------------------+
+-------------+-------------+
            |
            v
+---------------------------+
| Observability             |
| - Logs/Metrics/Traces     |
| - Audit log               |
+---------------------------+
```

3. Основные sequence flows

3.1 Flow A — Generate

```
User(UI)      Orchestrator      Storage      LLM Gateway      Renderer
  | POST /deck/generate      |            |            |            |
  |------------------------->|            |            |            |
  | create Deck, Run         |            |            |            |
  |-----> DB                 |            |            |            |
  | upload sources ----------> (S3)       |            |            |
  | Step P0 normalize -------------------->|            |            |
  | Step P2 storyline -------------------->|            |            |
  | Step P3 blueprint -------------------->|            |            |
  | Step P4 copy+notes ------------------->|            |            |
  | Step P6 QA/fix ----------------------->|            |            |
  | Step P7 render_spec ------------------>|            |            |
  | store slides/spec -> DB/S3             |            |            |
  |<------------------------- return deck_id + status     |            |
```

Принцип anti-hallucination

LLM Gateway и Orchestrator запрещают появление «новых фактов»: факты/цифры должны быть из Source, иначе помечаются NEEDS_SOURCE и/или запрашиваются у пользователя.

3.2 Flow B — Edit (частичная регенерация)

```
User(UI)      Orchestrator      Storage      LLM Gateway
  | POST /deck/{id}/edit    |            |             |
  |------------------------->| load context -> DB/S3      |
  | Step targeted_rewrite ---------------->|             |
  | validate brand-lock, length, QA       |             |
  | store new slide version -> DB         |             |
  |<------------------------- updated slide + version_id   |
```

3.3 Flow C — Export (PPTX/PDF)

```
User(UI)      Orchestrator      Storage        Renderer
  | POST /deck/{id}/export   |            |             |
  |------------------------->| create ExportJob -> DB      |
  | enqueue job ----------------------------------------> |
  |                        | fetch spec+tpl <------------ |
  |                        | render PPTX/PDF ------------>|
  |                        | store file -> S3 <---------- |
  |<------------------------- download_url / status        |
```

Идемпотентность экспорта: повторный экспорт по той же версии возвращает кэшированный результат.

4. Выбор технологий (варианты + почему)

Компонент	Рекомендуемый выбор / альтернативы
UI	React/Next.js (MVP): быстрый UX, удобный редактор. Альтернатива: Vue/Nuxt.
Orchestrator API	Python/FastAPI (MVP): удобная LLM-оркестрация и валидация схем. Альтернатива: Node.js/NestJS.
Очереди/асинхронные jobs	Redis + Celery/RQ (Python) или BullMQ (Node). Альтернатива: Kafka/RabbitMQ (сложнее для MVP).
Storage	PostgreSQL для сущностей/версий/аудита. S3-compatible (MinIO) для файлов и артефактов.
Renderer	PptxGenJS (Node) — предсказуемая верстка и экспорт. Альтернатива: python-pptx (часто сложнее с layout).
LLM Gateway	Тонкий сервис: routing + schema validation + caching + redaction.

	Централизует политику качества/стоимости/безопасности.
Observability	Prometheus+Grafana, Loki/ELK, OpenTelemetry. Audit: Postgres + append-only выгрузка (опционально).

5. Хранение и версионирование промптов

Хранение

Промпты — файлы в репозитории (/prompts) + метаданные (id, version, schema).

Версионирование

SemVer для promptset_version; prompt_id@prompt_version для шагов.

Воспроизводимость

GenerationRun сохраняет: promptset_version, prompt_version_per_step, model_id_per_step, input_hash, output_artifacts_ref (S3).

Контроль

JSON schema validation на выходе каждого шага; canary runs на тест-брифах при изменениях.

6. Безопасность (PII/ПДн, секреты, RBAC, audit log)

PII/ПДн

- Минимизация атрибутов (id, role, org_unit).
- Redaction ПДн в логах шагов LLM.
- Настраиваемая политика хранения данных per-tenant.

Secrets

- Secrets Manager (Vault/KMS/аналог).
- Запрет логирования секретов; доступ через сервисные аккаунты.

RBAC

- Роли: Student, Teacher, Admin, Buyer.
- Ограничения по org_unit и tenant isolation.

Audit log

- События: login, generate_start/finish, edit, export, template_update, role_change.
- Append-only хранение: Postgres + (опц.) write-once bucket/таблица без UPDATE прав.

7. Масштабирование (MVP → growth)

- Renderer и LLM-пайплайн масштабируются горизонтально воркерами.
- LLM Gateway централизует лимиты стоимости и качество.

- Multi-tenant изоляция через tenant_id на всех сущностях и отдельные префиксы/buckets в S3.

Связанные документы: PRD, SRS, Prompt Architecture, API Spec, QA Plan.